2.3 런타임 구성 및 개발 가이드

개요

Substrate 런타임은 블록체인의 상태 전환 함수(State Transition Function)를 정의하는 핵심 컴포넌트입니다. 런타임은 FRAME 팔렛들을 조합하여 구성되며, WebAssembly로 컴파일되어 온체인에서 실행됩니다. 이 문서에서는 Substrate 런타임의 아키텍처, 구성 방법, 그리고 개발 과정을 상세히 설명합니다.

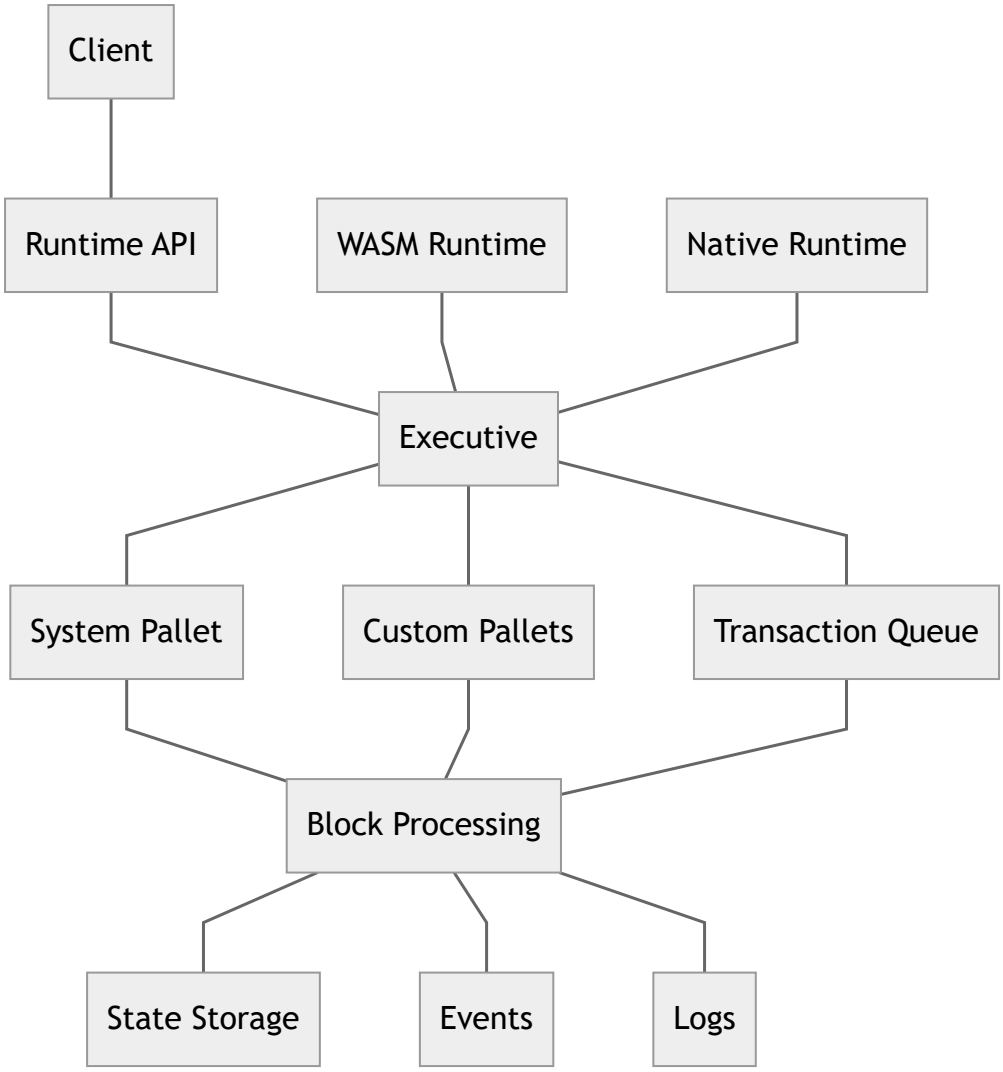
Substrate 런타임 아키텍처

런타임의 역할

런타임은 다음과 같은 핵심 기능을 담당합니다:

- **상태 전환**: 트랜잭션을 처리하여 블록체인 상태를 업데이트
- **블록 검증**: 새로운 블록의 유효성 검증
- **API 제공**: 클라이언트가 런타임과 상호작용할 수 있는 인터페이스 제공
- **업그레이드**: 포크 없이 런타임 로직 업데이트

런타임 아키텍처 다이어그램



런타임 구성 방법

1. 기본 런타임 구조

Substrate 런타임은 다음과 같은 기본 구조를 가집니다:

```
// runtime/src/lib.rs

#![cfg_attr(not(feature = "std"), no_std)]



extern  crate alloc;
```

```
use alloc::vec::Vec;

use sp_runtime::{

generic, impl_opaque_keys,

traits::{BlakeTwo256, IdentifyAccount, Verify},

MultiAddress, MultiSignature,

};



// 런타임 버전 정의

#[sp_version::runtime_version]

pub  const VERSION: RuntimeVersion = RuntimeVersion {

spec_name: alloc::borrow::Cow::Borrowed("my-runtime"),

impl_name: alloc::borrow::Cow::Borrowed("my-runtime"),

authoring_version: 1,

spec_version: 100,

impl_version: 1,

apis: RUNTIME_API_VERSIONS,

transaction_version: 1,

system_version: 1,

};



// 기본 타입 정의

pub  type Signature = MultiSignature;

pub  type AccountId = <<Signature as Verify>::Signer as IdentifyAccount>::Accoun
```

```
pub type Balance = u128;

pub type Nonce = u32;

pub type Hash = sp_core::H256;

pub type BlockNumber = u32;

pub type Address = MultiAddress<AccountId, ()>;

pub type Header = generic::Header<BlockNumber, BlakeTwo256>;

pub type Block = generic::Block<Header, UncheckedExtrinsic>;
```

2. 런타임 구성 매크로

최신 Substrate에서는 `#[frame_support::runtime]` 매크로를 사용하여 런타임을 구성합니다:

```
#[frame_support::runtime]

mod runtime {

#[runtime::runtime]

#[runtime::derive(

RuntimeCall,

RuntimeEvent,

RuntimeError,

RuntimeOrigin,

RuntimeFreezeReason,

RuntimeHoldReason,

RuntimeSlashReason,

RuntimeLockId,
```

```
RuntimeTask,
RuntimeViewFunction
)]
pub struct Runtime;

// 팔렛 인덱스와 함께 팔렛 등록
#[runtime::pallet_index(0)]
pub type System = frame_system;

#[runtime::pallet_index(1)]
pub type Timestamp = pallet_timestamp;

#[runtime::pallet_index(2)]
pub type Balances = pallet_balances;

#[runtime::pallet_index(3)]
pub type TransactionPayment = pallet_transaction_payment;

// 커스텀 팔렛 추가
#[runtime::pallet_index(10)]
pub type MyPallet = pallet_my_pallet;
}
```

3. 팔렛 설정 구현

각 팔렛에 대해 `Config` 트레이트를 구현해야 합니다:

```
// System 팔렛 설정
impl frame_system::Config for Runtime {
type BaseCallFilter = frame_support::traits::Everything;
type BlockWeights = BlockWeights;
type BlockLength = BlockLength;
type DbWeight = RocksDbWeight;
type RuntimeOrigin = RuntimeOrigin;
type RuntimeCall = RuntimeCall;
type Nonce = Nonce;
type Hash = Hash;
type Hashing = BlakeTwo256;
type AccountId = AccountId;
type Lookup = AccountIdLookup<AccountId, ()>;
type Block = Block;
type RuntimeEvent = RuntimeEvent;
type BlockHashCount = BlockHashCount;
type Version = Version;
type PalletInfo = PalletInfo;
type AccountData = pallet_balances::AccountData<Balance>;
type OnNewAccount = ();
```

```
type OnKilledAccount = ();

type SystemWeightInfo = ();

type SS58Prefix = SS58Prefix;

type OnSetCode = ();

type MaxConsumers = ConstU32<16>;

}



// Balances 팔렛 설정

impl pallet_balances::Config for Runtime {

type MaxLocks = ConstU32<50>;

type MaxReserves = ();

type ReserveIdentifier = [u8; 8];

type Balance = Balance;

type RuntimeEvent = RuntimeEvent;

type DustRemoval = ();

type ExistentialDeposit = ExistentialDeposit;

type AccountStore = System;

type WeightInfo = pallet_balances::weights::SubstrateWeight<Runtime>;

type FreezeIdentifier = RuntimeFreezeReason;

type MaxFreezes = ConstU32<0>;

type RuntimeHoldReason = RuntimeHoldReason;

type RuntimeFreezeReason = RuntimeFreezeReason;

}
```

```
// 커스텀 팔렛 설정
impl pallet_my_pallet::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type Currency = Balances;
type MaxItems = ConstU32<100>;
type WeightInfo = pallet_my_pallet::weights::SubstrateWeight<Runtime>;
}
```

4. 상수 및 매개변수 정의

런타임에서 사용할 상수들을 정의합니다:

```
use frame_support::{
parameter_types,
traits::{ConstU32, ConstU64, ConstU128},
weights::{Weight, constants::WEIGHT_REF_TIME_PER_SECOND},
};

// 블록 시간 관련 상수
pub const MILLI_SECS_PER_BLOCK: u64 = 6000;
pub const SLOT_DURATION: u64 = MILLI_SECS_PER_BLOCK;

// 시간 단위
```

```
pub const MINUTES: BlockNumber = 60_000 / (MILLI_SECS_PER_BLOCK as BlockNumber)
pub const HOURS: BlockNumber = MINUTES * 60;
pub const DAYS: BlockNumber = HOURS * 24;

// 잔액 단위
pub const UNIT: Balance = 1_000_000_000_000;
pub const MILLI_UNIT: Balance = 1_000_000_000;
pub const EXISTENTIAL_DEPOSIT: Balance = MILLI_UNIT;

// 매개변수 타입 정의
parameter_types! {
pub const BlockHashCount: BlockNumber = 2400;
pub const Version: RuntimeVersion = VERSION;
pub const SS58Prefix: u8 = 42;
pub const ExistentialDeposit: Balance = EXISTENTIAL_DEPOSIT;
}

// 블록 가중치 설정
parameter_types! {
pub const MaximumBlockWeight: Weight = Weight::from_parts(
WEIGHT_REF_TIME_PER_SECOND.saturating_mul(2),
u64::MAX,
);
```

```rust
pub const MaximumBlockLength: u32 = 5 * 1024 * 1024;

pub const AvailableBlockRatio: Perbill = Perbill::from_percent(75);

}


pub type BlockWeights = frame_system::limits::BlockWeights;

pub type BlockLength = frame_system::limits::BlockLength;
```

런타임 개발 과정

1. 프로젝트 구조 설정

```
my-runtime/

├── Cargo.toml

├── src/

| ├── lib.rs # 메인 런타임 파일

| ├── configs.rs # 팔렛 설정들

| ├── apis.rs # 런타임 API 구현

| └── weights.rs # 가중치 정의

├── build.rs # 빌드 스크립트

└── README.md
```

2. Cargo.toml 설정

```toml
[package]
```

```
name = "my-runtime"

version = "1.0.0"

edition = "2021"



[dependencies]

# Substrate primitives

sp-api = { workspace = true, default-features = false }

sp-block-builder = { workspace = true, default-features = false }

sp-consensus-aura = { workspace = true, default-features = false }

sp-consensus-grandpa = { workspace = true, default-features = false }

sp-core = { workspace = true, default-features = false }

sp-inherents = { workspace = true, default-features = false }

sp-offchain = { workspace = true, default-features = false }

sp-runtime = { workspace = true, default-features = false }

sp-session = { workspace = true, default-features = false }

sp-std = { workspace = true, default-features = false }

sp-storage = { workspace = true, default-features = false }

sp-transaction-pool = { workspace = true, default-features = false }

sp-version = { workspace = true, default-features = false }



# FRAME dependencies

frame-executive = { workspace = true, default-features = false }

frame-support = { workspace = true, default-features = false }
```

```
frame-system = { workspace = true, default-features = false }

frame-try-runtime = { workspace = true, default-features = false, optional = tru



# FRAME pallets

pallet-aura = { workspace = true, default-features = false }

pallet-balances = { workspace = true, default-features = false }

pallet-grandpa = { workspace = true, default-features = false }

pallet-sudo = { workspace = true, default-features = false }

pallet-timestamp = { workspace = true, default-features = false }

pallet-transaction-payment = { workspace = true, default-features = false }



# 커스텀 팔렛

pallet-my-pallet = { path = "../pallets/my-pallet", default-features = false }



[build-dependencies]

substrate-wasm-builder = { workspace = true, optional = true }



[features]

default = ["std"]

std = [

"sp-api/std",

"sp-block-builder/std",

"sp-consensus-aura/std",
```

```
"sp-consensus-grandpa/std",
"sp-core/std",
"sp-inherents/std",
"sp-offchain/std",
"sp-runtime/std",
"sp-session/std",
"sp-std/std",
"sp-storage/std",
"sp-transaction-pool/std",
"sp-version/std",
"frame-executive/std",
"frame-support/std",
"frame-system/std",
"frame-try-runtime?/std",
"pallet-aura/std",
"pallet-balances/std",
"pallet-grandpa/std",
"pallet-sudo/std",
"pallet-timestamp/std",
"pallet-transaction-payment/std",
"pallet-my-pallet/std",
"substrate-wasm-builder",
]
```

```
runtime-benchmarks = [
"frame-benchmarking/runtime-benchmarks",
"frame-support/runtime-benchmarks",
"frame-system/runtime-benchmarks",
"pallet-balances/runtime-benchmarks",
"pallet-my-pallet/runtime-benchmarks",
]

try-runtime = [
"frame-try-runtime/try-runtime",
"frame-executive/try-runtime",
"frame-system/try-runtime",
"pallet-balances/try-runtime",
"pallet-my-pallet/try-runtime",
]
```

3. 빌드 스크립트 (build.rs)

```rust
#[cfg(feature = "std")]
fn  main() {
substrate_wasm_builder::WasmBuilder::new()
.with_current_project()
```

```
.export_heap_base()

.import_memory()

.build()

}


#[cfg(not(feature = "std"))]

fn main() {}
```

4. 런타임 API 구현

```
// apis.rs

use sp_api::impl_runtime_apis;

use sp_runtime::traits::Block as BlockT;


impl_runtime_apis! {

impl sp_api::Core<Block> for Runtime {

fn version() -> RuntimeVersion {

VERSION

}


fn execute_block(block: Block) {

Executive::execute_block(block);

}
```

```
fn  initialize_block(header: &<Block as BlockT>::Header) -> sp_runtime::Extrinsi
Executive::initialize_block(header)
}
}

impl sp_api::Metadata<Block> for Runtime {
fn  metadata() -> OpaqueMetadata {
OpaqueMetadata::new(Runtime::metadata().into())
}

fn  metadata_at_version(version: u32) -> Option<OpaqueMetadata> {
Runtime::metadata_at_version(version)
}

fn  metadata_versions() -> sp_std::vec::Vec<u32> {
Runtime::metadata_versions()
}
}

impl sp_block_builder::BlockBuilder<Block> for Runtime {
fn  apply_extrinsic(extrinsic: <Block as BlockT>::Extrinsic) -> ApplyExtrinsicRe
Executive::apply_extrinsic(extrinsic)
```

```
}

fn  finalize_block() -> <Block as BlockT>::Header {
Executive::finalize_block()
}

fn  inherent_extrinsics(data: sp_inherents::InherentData) -> Vec<<Block as Block
data.create_extrinsics()
}

fn  check_inherents(
block: Block,
data: sp_inherents::InherentData,
) -> sp_inherents::CheckInherentsResult {
data.check_extrinsics(&block)
}
}

impl sp_transaction_pool::runtime_api::TaggedTransactionQueue<Block> for Runtime
fn  validate_transaction(
source: TransactionSource,
tx: <Block as BlockT>::Extrinsic,
block_hash: <Block as BlockT>::Hash,
```

```
) -> TransactionValidity {
Executive::validate_transaction(source, tx, block_hash)
}
}

impl sp_consensus_aura::AuraApi<Block, AuraId> for Runtime {
fn  slot_duration() -> sp_consensus_aura::SlotDuration {
sp_consensus_aura::SlotDuration::from_millis(Aura::slot_duration())
}

fn  authorities() -> Vec<AuraId> {
Aura::authorities().into_inner()
}
}

impl sp_consensus_grandpa::GrandpaApi<Block> for Runtime {
fn  grandpa_authorities() -> sp_consensus_grandpa::AuthorityList {
Grandpa::grandpa_authorities()
}

fn  current_set_id() -> sp_consensus_grandpa::SetId {
Grandpa::current_set_id()
}
```

```
fn  submit_report_equivocation_unsigned_extrinsic(
_equivocation_proof: sp_consensus_grandpa::EquivocationProof<
<Block as BlockT>::Hash,
NumberFor<Block>,
>,
_key_owner_proof: sp_consensus_grandpa::OpaqueKeyOwnershipProof,
) -> Option<()> {
None
}


fn  generate_key_ownership_proof(
_set_id: sp_consensus_grandpa::SetId,
_authority_id: GrandpaId,
) -> Option<sp_consensus_grandpa::OpaqueKeyOwnershipProof> {
None
}
}


impl frame_system_rpc_runtime_api::AccountNonceApi<Block, AccountId, Nonce> for
fn  account_nonce(account: AccountId) -> Nonce {
System::account_nonce(account)
}
```

```
}

impl pallet_transaction_payment_rpc_runtime_api::TransactionPaymentApi<Block, Ba

fn  query_info(

uxt: <Block as BlockT>::Extrinsic,

len: u32,

) -> pallet_transaction_payment_rpc_runtime_api::RuntimeDispatchInfo<Balance> {

TransactionPayment::query_info(uxt, len)

}


fn  query_fee_details(

uxt: <Block as BlockT>::Extrinsic,

len: u32,

) -> pallet_transaction_payment::FeeDetails<Balance> {

TransactionPayment::query_fee_details(uxt, len)

}


fn  query_weight_to_fee(weight: Weight) -> Balance {

TransactionPayment::weight_to_fee(weight)

}


fn  query_length_to_fee(length: u32) -> Balance {

TransactionPayment::length_to_fee(length)
```

```
}
}

#[cfg(feature = "runtime-benchmarks")]
impl frame_benchmarking::Benchmark<Block> for Runtime {
fn  benchmark_metadata(extra: bool) -> (
Vec<frame_benchmarking::BenchmarkList>,
Vec<frame_support::traits::StorageInfo>,
) {
use frame_benchmarking::{baseline, Benchmarking, BenchmarkList};
use frame_support::traits::StorageInfoTrait;
use frame_system_benchmarking::Pallet as SystemBench;
use baseline::Pallet as BaselineBench;

let  mut  list = Vec::<BenchmarkList>::new();
list_benchmarks!(list, extra);

let  storage_info = AllPalletsWithSystem::storage_info();

(list, storage_info)
}

fn  dispatch_benchmark(
```

```
config: frame_benchmarking::BenchmarkConfig

) -> Result<Vec<frame_benchmarking::BenchmarkBatch>, sp_runtime::RuntimeString>

use frame_benchmarking::{baseline, Benchmarking, BenchmarkBatch};

use sp_storage::TrackedStorageKey;

use frame_system_benchmarking::Pallet as SystemBench;

use baseline::Pallet as BaselineBench;



impl frame_system_benchmarking::Config for Runtime {}

impl baseline::Config for Runtime {}



use frame_support::traits::WhitelistedStorageKeys;

let  whitelist: Vec<TrackedStorageKey> = AllPalletsWithSystem::whitelisted_stora



let  mut  batches = Vec::<BenchmarkBatch>::new();

let  params = (&config, &whitelist);

add_benchmarks!(params, batches);



Ok(batches)

}

}



#[cfg(feature = "try-runtime")]

impl frame_try_runtime::TryRuntime<Block> for Runtime {
```

```
fn on_runtime_upgrade(checks: frame_try_runtime::UpgradeCheckSelect) -> (Weight

let weight = Executive::try_runtime_upgrade(checks).unwrap();

(weight, BlockWeights::get().max_block)

}



fn execute_block(

block: Block,

state_root_check: bool,

signature_check: bool,

select: frame_try_runtime::TryStateSelect,

) -> Weight {

Executive::try_execute_block(block, state_root_check, signature_check, select).e

}

}

}
```

런타임 업그레이드 메커니즘

1. 포크리스 업그레이드

Substrate는 하드포크 없이 런타임을 업그레이드할 수 있는 메커니즘을 제공합니다:

```
// 런타임 버전 업데이트

#[sp_version::runtime_version]
```

```
pub const VERSION: RuntimeVersion = RuntimeVersion {
spec_name: alloc::borrow::Cow::Borrowed("my-runtime"),
impl_name: alloc::borrow::Cow::Borrowed("my-runtime"),
authoring_version: 1,
spec_version: 101, // 업그레이드 시 증가
impl_version: 0, // spec_version 증가 시 0으로 리셋
apis: RUNTIME_API_VERSIONS,
transaction_version: 1,
system_version: 1,
};
```

2. 스토리지 마이그레이션

런타임 업그레이드 시 스토리지 구조가 변경되면 마이그레이션이 필요합니다:

```
use frame_support::{
traits::{OnRuntimeUpgrade, StorageVersion},
weights::Weight,
};

pub struct MigrateToV2<T>(sp_std::marker::PhantomData<T>);

impl<T: Config> OnRuntimeUpgrade for MigrateToV2<T> {
fn on_runtime_upgrade() -> Weight {
```

```
let current_version = Pallet::<T>::current_storage_version();
let onchain_version = Pallet::<T>::on_chain_storage_version();

if onchain_version == 1 && current_version == 2 {
// 마이그레이션 로직 실행
migrate_storage_v1_to_v2::<T>();
// 새 버전으로 업데이트
current_version.put::<Pallet<T>>();
T::DbWeight::get().reads_writes(100, 100)
} else {
Weight::zero()
}
}
}

fn migrate_storage_v1_to_v2<T: Config>() {
// 기존 스토리지에서 데이터 읽기
let old_data = OldStorageMap::<T>::drain();
// 새 형식으로 변환하여 저장
for (key, old_value) in old_data {
let new_value = convert_old_to_new(old_value);
NewStorageMap::<T>::insert(key, new_value);
}
```

```
}

// 팔렛에서 마이그레이션 적용
#[pallet::hooks]
impl<T: Config> Hooks<BlockNumberFor<T>> for Pallet<T> {
fn  on_runtime_upgrade() -> Weight {
MigrateToV2::<T>::on_runtime_upgrade()
}
}
```

3. 거버넌스 연동

런타임 업그레이드를 거버넌스 시스템과 연동할 수 있습니다:

```
// Democracy 팔렛을 통한 런타임 업그레이드
impl pallet_democracy::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type Currency = Balances;
type EnactmentPeriod = EnactmentPeriod;
type LaunchPeriod = LaunchPeriod;
type VotingPeriod = VotingPeriod;
type VoteLockingPeriod = EnactmentPeriod;
type MinimumDeposit = MinimumDeposit;
type ExternalOrigin = EnsureRootOrHalfCouncil;
```

```
type ExternalMajorityOrigin = EnsureRootOrHalfCouncil;

type ExternalDefaultOrigin = EnsureRoot<AccountId>;

type SubmitOrigin = EnsureSigned<AccountId>;

type FastTrackOrigin = EnsureRootOrTwoThirdsCouncil;

type InstantOrigin = EnsureRoot<AccountId>;

type InstantAllowed = frame_support::traits::ConstBool<true>;

type FastTrackVotingPeriod = FastTrackVotingPeriod;

type CancellationOrigin = EnsureRootOrThreeFourthsCouncil;

type BlacklistOrigin = EnsureRoot<AccountId>;

type CancelProposalOrigin = EnsureRootOrAllCouncil;

type VetoOrigin = pallet_collective::EnsureMember<AccountId, CouncilCollective>;

type CooloffPeriod = CooloffPeriod;

type Slash = Treasury;

type Scheduler = Scheduler;

type PalletsOrigin = OriginCaller;

type MaxVotes = ConstU32<100>;

type WeightInfo = pallet_democracy::weights::SubstrateWeight<Runtime>;

type MaxProposals = MaxProposals;

type Preimages = Preimage;

type MaxDeposits = ConstU32<100>;

type MaxBlacklisted = ConstU32<100>;

}
```

실제 런타임 구성 예제

간단한 DeFi 런타임

```
#[frame_support::runtime]
mod runtime {
#[runtime::runtime]
#[runtime::derive(
RuntimeCall,
RuntimeEvent,
RuntimeError,
RuntimeOrigin,
RuntimeFreezeReason,
RuntimeHoldReason,
RuntimeSlashReason,
RuntimeLockId,
RuntimeTask,
RuntimeViewFunction
)]
pub  struct Runtime;

// 기본 시스템 팔렛들
#[runtime::pallet_index(0)]
```

```rust
pub type System = frame_system;

#[runtime::pallet_index(1)]
pub type Timestamp = pallet_timestamp;

#[runtime::pallet_index(2)]
pub type Balances = pallet_balances;

#[runtime::pallet_index(3)]
pub type TransactionPayment = pallet_transaction_payment;

// 합의 관련 팔렛들
#[runtime::pallet_index(10)]
pub type Aura = pallet_aura;

#[runtime::pallet_index(11)]
pub type Grandpa = pallet_grandpa;

// DeFi 관련 팔렛들
#[runtime::pallet_index(20)]
pub type Assets = pallet_assets;

#[runtime::pallet_index(21)]
```

```rust
pub type AssetConversion = pallet_asset_conversion;


#[runtime::pallet_index(22)]
pub type Staking = pallet_staking;


// 거버넌스 팔렛들
#[runtime::pallet_index(30)]
pub type Democracy = pallet_democracy;


#[runtime::pallet_index(31)]
pub type Council = pallet_collective;


#[runtime::pallet_index(32)]
pub type Treasury = pallet_treasury;


// 유틸리티 팔렛들
#[runtime::pallet_index(40)]
pub type Utility = pallet_utility;


#[runtime::pallet_index(41)]
pub type Multisig = pallet_multisig;


#[runtime::pallet_index(42)]
```

```
pub type Proxy = pallet_proxy;


// 커스텀 DeFi 팔렛
#[runtime::pallet_index(50)]
pub type DeFiPool = pallet_defi_pool;
}
```

런타임 테스트 방법

1. 단위 테스트

```
#[cfg(test)]
mod tests {
use super::*;
use frame_support::{
assert_ok, assert_noop,
traits::{OnInitialize, OnFinalize},
weights::Weight,
};
use sp_runtime::BuildStorage;


fn new_test_ext() -> sp_io::TestExternalities {
let mut t = frame_system::GenesisConfig::<Runtime>::default()
```

```
.build_storage()
.unwrap();
pallet_balances::GenesisConfig::<Runtime> {
balances: vec![(1, 100), (2, 100)],
}
.assimilate_storage(&mut t)
.unwrap();

t.into()
}

#[test]
fn basic_setup_works() {
new_test_ext().execute_with(|| {
assert_eq!(Balances::free_balance(1), 100);
assert_eq!(Balances::free_balance(2), 100);
});
}

#[test]
fn transfer_works() {
new_test_ext().execute_with(|| {
assert_ok!(Balances::transfer_allow_death(
```

```
RuntimeOrigin::signed(1),
2,
50
));
assert_eq!(Balances::free_balance(1), 50);
assert_eq!(Balances::free_balance(2), 150);
});
}
}
```

2. 통합 테스트

```
#[cfg(test)]
mod integration_tests {
use  super::*;
use frame_support::traits::Hooks;

#[test]
fn  block_production_works() {
new_test_ext().execute_with(|| {
// 블록 초기화
System::set_block_number(1);
AllPalletsWithSystem::on_initialize(1);
```

```
// 트랜잭션 실행
assert_ok!(Balances::transfer_allow_death(
RuntimeOrigin::signed(1),
2,
10
));

// 블록 완료
AllPalletsWithSystem::on_finalize(1);
// 상태 검증
assert_eq!(System::block_number(), 1);
assert_eq!(Balances::free_balance(1), 90);
});
}
}
```

성능 최적화 및 베스트 프랙티스

1. 가중치 최적화

```
// 벤치마킹 기반 가중치 사용
#[pallet::weight(T::WeightInfo::transfer())]
```

```
pub fn transfer(
origin: OriginFor<T>,
dest: T::AccountId,
amount: T::Balance,
) -> DispatchResult {
// 구현...
}


// 동적 가중치 계산
#[pallet::weight(T::WeightInfo::batch(calls.len() as u32))]
pub fn batch(
origin: OriginFor<T>,
calls: Vec<Box<<T as Config>::RuntimeCall>>,
) -> DispatchResult {
// 구현...
}
```

2. 스토리지 최적화

```
// BoundedVec 사용으로 스토리지 크기 제한
#[pallet::storage]
pub type Items<T: Config> = StorageMap<
_,
```

```
Blake2_128Concat,
T::AccountId,
BoundedVec<u32, T::MaxItems>,
ValueQuery
>;


// 효율적인 해시 함수 선택
#[pallet::storage]
pub  type FastLookup<T: Config> = StorageMap<
_,
Twox64Concat, // 빠른 해시, 신뢰할 수 있는 키에만 사용
u32,
T::AccountId,
OptionQuery
>;


#[pallet::storage]
pub  type SecureLookup<T: Config> = StorageMap<
_,
Blake2_128Concat, // 보안 해시, 사용자 입력 키에 사용
T::AccountId,
u32,
OptionQuery
```

```
>;
```

3. 메모리 관리

```
// 대용량 데이터 처리 시 청크 단위로 처리
#[pallet::call]
impl<T: Config> Pallet<T> {
#[pallet::weight(T::WeightInfo::process_batch(items.len() as u32))]
pub fn process_batch(
origin: OriginFor<T>,
items: Vec<u32>,
) -> DispatchResult {
let _who = ensure_signed(origin)?;
// 청크 단위로 처리하여 메모리 사용량 제한
for chunk in items.chunks(100) {
for &item in chunk {
Self::process_item(item)?;
}
}
Ok(())
}
}
```

다음 단계

이 문서에서 다룬 런타임 구성과 개발 방법을 바탕으로, 다음 문서에서는 합의 메커니즘의 구현과 동작 원리를 자세히 살펴보겠습니다:

- 2.4 합의 메커니즘 (BABE, GRANDPA) 구현 분석